Sherlock — Team Brief

Part 1: the plan · Part 2: stretch goals and what others have done · Draft for discussion

9 October 2026

Part 1 — What we're building and how it breaks down

1. What Sherlock is

A local developer tool for teams that build multi-agent LLM systems on LangGraph. When a team's run produces a wrong answer, Sherlock re-runs the team with targeted fixes and reports **which agent — or which combination of agents — has to change** for the run to pass. If it cannot test that faithfully, it says "can't tell" instead of guessing.

Why it's needed

- In multi-agent pipelines the mistake usually happens several agents upstream, while every step still reports "OK" (described publicly by Anthropic's and Sentry's engineering teams).
- Reading the trace doesn't work well: the best published method on the Who&When benchmark finds the exact failing step only 14.2% of the time.
- Existing tools (LangSmith, Langfuse, Arize Phoenix) record runs but don't tell you what to fix, and most research methods blame a single step even when two agents are jointly at fault.

2. The GCJR paper and what it changes for us

The paper. Li et al., *Localizing Emergent Failures in Agentic AI: Recovering Minimal Repair Families via Counterfactual Replay*, arXiv 2608.29228 — posted 6 September 2026, shortly after our proposal was approved.

What it does

- Replays a failed multi-agent run with individual messages swapped for correct ("clean") versions, and returns **every minimal set of swaps that makes the run succeed** — so it can distinguish "A **and** B must both be fixed" from "**either** A **or** B fixes it".
- Uses the run's dependency graph to prune the search: same answer as trying every combination, with 52–55% fewer replays.
- Evaluated on 90 synthetic workflows and a 24-case arithmetic pilot with a small local model (Qwen2.5-1.5B); all four-agent setups.

What it took from our proposal. Our headline claim — "the smallest set of agents whose repair fixes the failure, verified by re-running the team" — is essentially GCJR's formulation. We can no longer present it as new.

What it does not do — and where Sherlock comes in

	GCJR	Sherlock
What a trial fixes	Overwrites one **message**	Restores an **agent's behaviour** (prompt, model, tool), which then recomputes its output

	GCJR	Sherlock
Two faults on the same chain (e.g. Retriever → Writer, both broken)	Out of scope — abstains on all 30 such test cases	Supported (example below)
Where the "correct" version comes from	Assumed to be given	A passing run, a pasted answer, or an earlier agent version — "can't tell" if none exists
Dependency graph	Assumed given; accuracy falls from 1.000 to 0.552 when 10% of edges are missing	Recovered from LangGraph's own checkpoints
Real framework / usable tool	None — research prototype	LangGraph integration, CLI + dashboard, installable

Why the chained case matters. Suppose the Retriever leaves out a key fact *and* the Writer independently inserts a false claim. Overwriting the Writer's *message* with a clean one fixes the output on its own, so message-level replay blames only the Writer — yet fixing the Writer's prompt alone would not fix the system. Restoring *behaviour* instead: fix Writer only → still fails (it writes from incomplete documents); fix Retriever only → still fails (the false claim remains); fix both → passes. Sherlock reports **{Retriever, Writer}** — what the engineer actually has to change.

How this changes the project

- **Positioning.** The FAST Idea Selection Guide explicitly accepts both "implementing a state-of-the-art method as a deployable product" and "improving it at a gap the group identified, with the original as baseline." Sherlock does both: GCJR's method becomes the engine of a real LangGraph tool, agent-level restoration is our extension, and GCJR is the baseline we compare against.
- **One proposal promise has to change.** We promised "no live model calls during diagnosis unless already recorded." That makes most trials impossible, because a broken agent's reaction to a *corrected* input was never recorded. GCJR re-runs live with repeated trials; we propose the same, using a local fixed-seed model and 3 repeats per trial.

3. How a diagnosis works

1. The engineer picks a failed run and states what "correct" means (e.g. the answer must contain X and must not mention Y).
2. Sherlock maps which agent fed which, and keeps only the agents that could have affected the final answer.
3. For each candidate agent, then each pair, it rewinds the run to just before that agent using LangGraph's checkpoints, restores the known-good version, and lets everything downstream re-run.
4. Each trial is repeated (default 3×); the check decides pass or fail.
5. Output: every minimal set of agents that fixes the run, with the trial log as proof — or "can't tell", with the reason.

4. How the project breaks down

Module	What it does	Status
M1 Ingestion	Connect to the team's LangGraph server; list teams, runs and per-agent history	**Done** (Iteration 1)
M2 Run Explorer	CLI and dashboard to browse runs and see each agent's output	Timeline done; graph view to do
M3 Outcome Checks	Define "correct": exact match, contains / must-not-contain, regex, JSON schema, custom Python	Next
M4 Replay Engine	Rewind a run at a checkpoint, apply a fix, re-run downstream, repeat	Next — riskiest part

Module	What it does	Status
M5 Diagnosis Engine	GCJR-style search → minimal sets (AND / OR) or "can't tell"	Next (message-level), then agent-level
M6 Known-Good Sources	Passing runs, pasted answers, earlier agent versions	Later
M7 Report & Sharing	Result page, trial log, comparison with naive methods, read-only share, save as regression test	Later
M8 Python Helper	A few lines in the team's code so Sherlock can swap an agent's prompt/model for a trial and block real side effects (e-mails, payments) during replays	Later

Where we are. Iteration 1 (Ali's repo) is a Rust CLI and local web dashboard that connects to a LangGraph server and shows teams, runs and an error-first timeline for each run, with about 40 automated tests. No diagnosis yet.

Architecture in one line. One `sherlock` binary (Rust core + embedded React dashboard) talks to the team's LangGraph Agent Server over HTTP; LangGraph's own checkpoints serve as the recording; the small Python helper (M8) is the only thing a team adds to its code.

5. Use cases

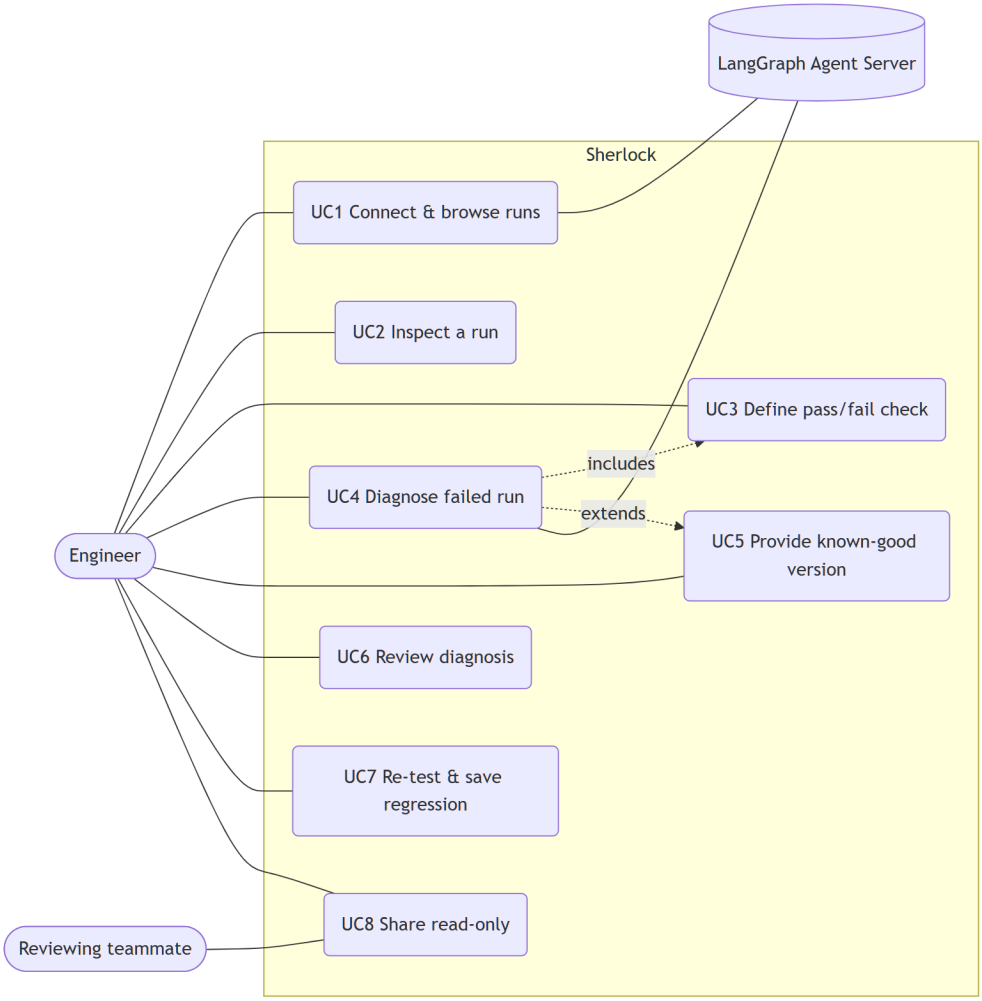


Figure 1: Use case diagram

UC	In one line
UC1 Connect & browse runs	Point Sherlock at the team's LangGraph server; list teams and runs
UC2 Inspect a run	See what each agent produced, errors first
UC3 Define pass/fail check	State what a correct final answer looks like
UC4 Diagnose failed run	Run the trials and get the minimal set(s) of agents to fix, or "can't tell"
UC5 Provide known-good version	Supply a passing run, a pasted answer or an earlier agent version
UC6 Review diagnosis	Read the result and the trial log that proves it
UC7 Re-test & save regression	After fixing the code, confirm the fix and keep the case as a test
UC8 Share read-only	Send a teammate a view that cannot re-run anything

6. Functional requirements

ID	Requirement	Priority
FR1	Connect to a LangGraph Agent Server (URL + optional API key) and list teams and runs	Must — done
FR2	Show a run's per-agent history (outputs, errors) in CLI and dashboard	Must — done
FR3	Show the run as a graph of agents and handoffs	Should
FR4	Define a pass/fail check on the final output: exact, contains, must-not-contain, regex	Must
FR5	JSON-schema and custom Python checks	Should
FR6	Re-run from any checkpoint with one agent's output or behaviour replaced, without altering the original run	Must
FR7	Repeat each trial N times; decide pass / fail / inconclusive by threshold	Must
FR8	Keep only agents that can affect the final output as candidates	Must
FR9	Test single agents, then pairs; return every minimal fixing set	Must
FR10	Allow pairs on the same chain (agent-level mode)	Must
FR11	Return "can't tell", with the reason, when no known-good version exists or trials are inconclusive	Must
FR12	Accept known-good versions from a passing run or a pasted answer; from an earlier agent configuration	Must; Should
FR13	An LLM may suggest which agents to try first, but never decides the cause	Must
FR14	Never execute side-effecting tools during replay; use recorded results or abort the trial	Must
FR15	Show the result with its trial log and cost (calls, time)	Must
FR16	Compare against naive methods (single-agent replay, message-level GCJR) on the same run	Should
FR17	Export a read-only share view	Should
FR18	Save a diagnosed run as a regression test and re-test after a fix	Should

7. Non-functional requirements

Quality	Requirement
Reliability	Same inputs give the same result (10/10 repeats on the demo team with a local model); the original run's history is never modified
Performance	Diagnosing the three-agent demo takes ≤ 5 minutes on our RTX 5070 with a local model; ≥ 40% fewer replays than trying every combination

Quality	Requirement
Usability	Someone outside the team installs Sherlock and diagnoses the demo in ≤ 15 minutes from the README; every “can’t tell” is explained in one sentence
Security & privacy	Runs locally and binds to localhost by default; API keys never logged; secrets and personal data masked before anything is saved or shared
Portability	Single binary for Windows, macOS and Linux; a pinned, documented LangGraph version range

8. Open questions for the team

1. Do we accept the repositioning — GCJR as the engine, agent-level restoration as our extension, GCJR as baseline?
2. Do we accept replacing the “no live calls” promise with repeated live trials on a local model?
3. What is the minimum product we must ship — M1–M5 plus a basic M7?
4. Is agent-level diagnosis (M8) the FYP-2 headline, or do we pull it into FYP-1 so mid-eval shows something GCJR can’t do?
5. Who owns which module?

Part 2 — Stretch goals and what other researchers have done

> **This is Part 2.** Part 1 is the plan we commit to. Part 2 is for discussion: the few papers that actually matter for what we build, the stretch goals we could add, and a concrete list of additions (§13). Nothing here is a commitment yet.

9. How to read this part

We read 22 papers around GCJR and kept only the **10 that change what we build**. (Notes on all 22 are in `report/research/` if anyone wants more. No paper cites GCJR yet.)

Word	Meaning here
Run	One execution of the agent team, with everything each agent received and produced
Re-run from a checkpoint	Restart the team from a saved point in the middle, so only the agents after it run again
Fix	The one change we try in a re-run — e.g. give an agent its correct output, or its old prompt
Known-good version	Where the correct output or prompt comes from: a passing run, a pasted answer, an older version
Planted fault	Breaking an agent on purpose so we know the right answer when testing Sherlock

10. The field in one picture

Approach	How it decides who is to blame	Papers below
Read the log	An LLM reads the run and guesses — right about the exact step only ~14–30% of the time on long runs	Who&When, SCOPE, MAScope
Re-run with one change	Changes one thing, re-runs, sees whether the failure goes away — trustworthy but costly	DoVer, SymTrace, CAR, HPFA
Predict from training	Learns from thousands of offline re-runs which setting to change	DAAF
Tools we can borrow from	Testing statistics and dashboard design	AgentAssay, PROTEA

The big picture: nobody ships an installable debugger that re-runs a team to find **every smallest set of agents** whose prompt or model must change. DAAF comes closest but *predicts* one change instead of checking.

11. The 10 papers that matter

Re-run based — closest to Sherlock

DoVer (Microsoft, 2026)

- **What it does:** an LLM guesses where a failed run went wrong; DoVer edits that step, re-runs the rest 3 times, and labels the guess *validated / partly validated / refuted / inconclusive*. It rescued 18–28% of failed runs; "inconclusive" was the most common result on hard tasks.
- **What we take:** these four verdicts for every suspect, and a check that the fix was really applied before calling an agent innocent. (It only edits messages and returns one suspect — our extension still stands.)

SymTrace — "Repair or Resample?" (2026)

- **What it does:** shows many "fixes" are luck. It replays the recorded start of a run exactly, then runs live from the suspect step. Plain re-running "fixed" only ~7% of failures, and **39 of 54 runs that had passed failed at least once when simply re-run.**
- **What we take:** before diagnosing, re-run the failed run unchanged a few times — if it sometimes passes, report it as *flaky*; always show pass counts ("2 of 3"), never one tick; stop with "replay went off track" instead of guessing.

Causal Agent Replay — CAR (CMU, 2026, open source)

- **What it does:** for a single agent, changes one step (re-draw it, replace a tool result, edit the prompt, or swap the model), re-runs many times, and compares failure rates with confidence ranges.
- **What we take:** its list of change types is a ready menu for Sherlock's fixes; confidence ranges on pass rates; its code (Apache-2.0) includes a LangGraph recorder and an HTML report we can study or adapt.

HPFA (COLM 2026)

- **What it does:** compares a failed run with a successful run of the same task and only tests "swap in the good version" where they differ. When a method only gets credit if its fix really flips the result, HPFA is about twice as good as log readers.
- **What we take:** a passing run of the same input is our best known-good source — compare agent by agent and test only agents whose output differs; make "how often our fix really makes the run pass" Sherlock's headline number.

Log reading — useful for ordering suspects only

Who&When (ICML 2025)

- **What it does:** the standard benchmark — 184 failed multi-agent runs labelled by humans. The best log-reading method names the right agent 53.5% of the time and the right step only 14.2%.
- **What we take:** report the same accuracy numbers so we can compare with published work; its definition of a cause ("fix it and the run passes") is exactly what Sherlock tests for real.

SCOPE (AAAI 2026)

- **What it does:** shortlists suspects with two simple rules before asking an LLM — an agent **stuck repeating itself**, or an agent **acting outside its role**. Step accuracy on long runs rose from 3.5% to 27.8%; the loop rule mattered most.
- **What we take:** a loop detector that needs no LLM, and "try the shortlist first, then widen" to save re-runs.

MAScope (October 2026)

- **What it does:** works out the team's shape (pipeline, hub-and-workers, …), which predicts how it tends to fail. Its dataset includes **408 LangGraph runs with planted faults**.
- **What we take:** read the team's shape straight from LangGraph and show it in the dashboard; use its LangGraph runs as test data (after checking the licence).

Tools we can borrow from

AgentAssay (2026, open-source library)

- **What it does:** runs an agent several times and answers *pass / fail / inconclusive* with a confidence range, stopping early once the answer is clear (22 runs on average instead of 100). Has a LangGraph adapter and CI exit codes.
- **What we take:** the maths for smart repeats (S2) and the CI pattern (S8). Apache-2.0.

PROTEA (Sony, ACL 2026 demo)

- **What it does:** a web tool that colours each agent in the workflow graph pass/warn/fail, opens a side panel with the agent's inputs and outputs, suggests a prompt edit as a before/after diff, re-runs, and allows rollback. Its colours come from an LLM's opinion, not from re-running.
- **What we take:** the dashboard layout — but colour agents by Sherlock's *verified* result (must change / not needed / can't tell); before/after diff with rollback for suggested fixes.

The closest competitor

DAAF (Tencent, September 2026)

- **What it does:** decides which saved setting of an agent system to change (prompt, knowledge file, tool setting…). It runs thousands of swap-and-re-run experiments offline, trains a small model on them, then *predicts* one change for a new failure. It picked the right setting 81% of the time (an LLM: 41%), and found that **changes interact**: only 11% of single swaps mattered alone, but 31% mattered in combination.
- **How Sherlock differs:** DAAF predicts after per-system training and returns one change; Sherlock re-runs and checks with no training, and returns every smallest set of agents. DAAF was tested on one customer-service agent; Sherlock targets multi-agent LangGraph teams.
- **What we take:** give each agent's prompt, model and tools an id, a version and a known-good alternative (feeds S1); its interaction numbers back our core idea of testing combinations of agents.

12. Stretch goals

Things we *could* add after the Part 1 "Must" items. Effort for one person: **S** ≈ 1–2 weeks, **M** ≈ 3–5 weeks, **L** ≈ 6+ weeks (rough guesses).

S1 — Regression mode: "what changed and broke it?" · M

- **What:** tests passed last week and fail today; Sherlock finds the smallest set of changes (prompts, models, tools, across agents) that caused it.
- **How:** our Python helper saves each agent's settings per version; list what differs between the last good version and today; re-run today's version with some changes undone, using the same "smallest set" search.
- **From:** DAAF, AgentAssay.

S2 — Smart repeats with a confidence range · S–M

- **What:** repeat a test only until the answer is clear, then show "passed 7 of 8 — very likely fixed" or "can't tell yet".
- **How:** a short statistics formula after each repeat decides whether to stop; a cap keeps cost bounded.
- **From:** AgentAssay, CAR.

S3 — An accurate "who used whose output" map · M

- **What:** a better map means fewer pointless re-runs and fewer missed causes.
- **How:** read the edges from LangGraph's graph; have our helper record which parts of the shared state each agent actually read; show the team's shape in the dashboard.
- **From:** MAScope; GCJR (its accuracy fell from 1.000 to 0.552 with 10% of the map missing).

S4 — Fixes that need three or more agents · S–M

- **What:** we test single agents and pairs; some failures may need three.
- **How:** after pairs, split the remaining suspects in halves and test each; stop at a fixed budget.
- **From:** DAAF's finding that changes interact.

S5 — Smarter order of suspects · S

- **What:** try the most likely agents first, so a diagnosis needs fewer runs.
- **How:** free rules — an agent stuck in a loop, an agent acting outside its role, an agent whose output differs from a passing run; optionally one cheap LLM call to rank the rest. The order affects speed only; the re-run still decides.
- **From:** SCOPE, HPFA.

S6 — Suggested fixes, tested before we show them · M

- **What:** propose a prompt edit for each blamed agent, re-run with it, and show whether it fixes this run *and* whether it breaks any saved passing runs.
- **How:** an LLM drafts the edit; Sherlock re-runs; shown as a before/after diff with rollback.
- **From:** PROTEA. **Risk:** the LLM must never decide the cause (FAST rule).

S7 — Planted-fault test suite for LangGraph · M–L

- **What:** LangGraph teams where we break agents on purpose — including two broken agents on one chain — to prove Sherlock finds the right answer.
- **How:** 4–6 small teams; a script that breaks an agent (bad prompt, weaker model, wrong tool result); add MAScope's 408 LangGraph runs and Who&When for comparison.
- **From:** MAScope, Who&When. **We need this for our own evaluation anyway.**

S8 — Run inside CI · M

- **What:** on every pull request, run the team's tests; if one fails, Sherlock diagnoses it and posts the result as a PR comment.
- **How:** a GitHub Action calling the `sherlock` CLI; exit codes pass / fail / can't tell.
- **From:** AgentAssay.

S9 — Checks on individual agents · M

- **What:** optional checks on one agent's output (e.g. "the Retriever must return a source link"), not only on the final answer.

- **How:** checks attach to an agent's output; Sherlock skips agents that clearly did their job and catches problems a passing final answer hides.

13. What we should add to Sherlock

A. Add to the core now — cheap, and every result becomes more trustworthy

1. **Check the failure is real first:** re-run the failed run unchanged a few times; if it sometimes passes, report "flaky" and how often. *(SymTrace, HPFA)*
2. **Four verdicts per suspect:** fixes it / partly helps / doesn't help / can't tell. *(DoVer, AgentAssay)*
3. **Counts, not ticks:** "passed 2 of 3", with a confidence range. *(CAR, AgentAssay)*
4. **Make sure the fix was really applied**, and stop with "replay went off track" if the re-run's start doesn't match the original. *(DoVer, SymTrace)*
5. **Prefer rule-based checks** (contains, regex, JSON shape, number limits); mark LLM-judged checks as noisy. *(CAR, AgentAssay)*
6. **Show each agent's input next to its output**, and the smallest fix first.

B. Dashboard: PROTEA-style graph plus side panel, with agents coloured by verified result, and the path from the cause to where the failure showed up.

C. How we test Sherlock: planted faults including two broken agents on one chain (S7); compare against fixing a random agent, fixing the last agent, a plain retry, and an LLM reading the log *(SymTrace, Who&When)*; headline number = how often Sherlock's answer really makes the run pass *(HPFA)*.

D. Code and data we can reuse (check licences first)

What	Use
CAR — github.com/jaineet17/causal-agent-replay (Apache-2.0)	LangGraph recorder and HTML report to study or adapt
AgentAssay — PyPI (Apache-2.0)	Statistics for smart repeats and CI
MAScope-Bench — huggingface.co/datasets/mascope/mascope-bench	408 LangGraph runs with planted faults
Who&When — github.com/mingyin1/Agents_Failure_Attribution	Comparison with published numbers

E. Stretch goals to pick (suggestion): do S7, S2 and S5's free rules anyway; make **S1 regression mode** the product headline; leave S6 and S4 for later.

F. One line on how Sherlock is different

> Other tools **guess** from the log or **predict** from training. Sherlock **re-runs the team and checks**, then returns every smallest set of agents whose fix makes the run pass — or says it can't tell. And it's a tool you install, not a research prototype.